First Order Logic Unification

```
class Variable:
    def __init__(self, name):
        self.name = name
    def __repr__(self):
        return self.name

class Constant:
    def __init__(self, name):
        self.name = name
    def __repr__(self):
        return self.name

class Predicate:
    def __init__(self, name, args):
        self.name = name
        self.args = args
    def __repr__(self):
        return f"{self.name}({', '.join(map(str, self.args))})"

def unify_oop(e1, e2, sub):
    if sub == "FAILURE":
        return "FAILURE"
    if isinstance(e1, Variable) and isinstance(e2, Variable) and e1.name ==
e2.name:
        return sub
    if isinstance(e1, Constant) and isinstance(e2, Constant) and e1.name ==
e2.name:
        return sub
    if isinstance(e1, Variable):
        return unify_variable(e1, e2, sub)
    if isinstance(e2, Variable):
        return unify_variable(e2, e1, sub)
    if isinstance(e1, Predicate) and isinstance(e2, Predicate):
```

```
        if e1.name != e2.name or len(e1.args) != len(e2.args):
            return "FAILURE"
        for a1, a2 in zip(e1.args, e2.args):
            sub = unify_oop(a1, a2, sub)
        return sub
    return "FAILURE"

def unify_variable(v, x, sub):
    if v.name in sub:
        return unify_oop(sub[v.name], x, sub)
    if isinstance(x, Variable) and x.name in sub:
        return unify_oop(v, sub[x.name], sub)
    sub[v.name] = x
    return sub

x = Variable("x")
y = Variable("y")
riya = Constant("Riya")
apple = Constant("Apple")

p1 = Predicate("Eats", [x, apple])
p2 = Predicate("Eats", [riya, y])
print(unify_oop(p1, p2, {}))
```

Output

```
{'x': Riya, 'y': Apple}
```